

Collaborative Study Platform

Project Proposal for UCS503

Submitted to: Ms. Nisha Thakur, Thapar Institute of Engineering and Technology

Team Members:

Shashvt (1024170449) — Anirudh Sharma (1024170430) — Lavish (1024170441)

August 14, 2026

1 Higher-order Goal / Secondary Framing

This project aims to improve collaborative and resource-driven learning among college students by reducing the time and effort required to find, organize, share, and access academic study material.

While students commonly rely on disconnected sources such as messaging groups, cloud drives, social-media communities, and personal collections for notes, books, and previous-year questions, the immediate engineering objective is to consolidate these activities into a single, structured, deployable web platform.

The platform focuses on making academic resources easier to discover while enabling students to contribute resources and participate in collaborative study activities.

2 Time-to-Value / Primary Heuristic

We will deliver meaningful increments quickly by first stabilizing the core student workflow: authentication, resource discovery, resource upload, and access to study materials.

We will prioritize rapid feedback over unnecessary complexity by implementing the platform in incremental iterations and validating each major workflow through actual usage.

Existing mature technologies and services will be used rather than developing infrastructure from scratch.

The initial implementation will focus on correctness of the core workflows before introducing advanced capabilities such as recommendation systems, analytics, or AI-assisted learning.

Success is defined by what can be delivered and measured in the first iteration, followed by improvements based on student feedback and observed usage.

3 Problem Statement

College students frequently need to access academic resources such as lecture notes, books, previous-year questions, and other study material. However, these resources are often distributed across multiple disconnected platforms.

Common pain points include:

- **Fragmentation:** Study material is distributed across WhatsApp groups, Telegram groups, Google Drive links, personal folders, and other sources, making resources difficult to locate.

- **Poor organization:** Students may have difficulty finding material according to academic year, stream, subject, or subject code.
- **Limited discoverability:** Useful notes or previous-year questions may exist within a student community but remain difficult for other students to discover.
- **Repeated sharing:** The same files are frequently shared multiple times across different groups, resulting in unnecessary duplication.
- **Lack of structured contribution:** Students who possess useful notes, books, or PYQs do not always have a centralized platform through which they can contribute them.
- **Limited collaboration:** Conventional LMS platforms generally focus on teacher-to-student content delivery rather than student-to-student resource sharing and study collaboration.
- **Resource accessibility:** Large academic files require appropriate storage and delivery infrastructure rather than being permanently stored inside the application itself.

Why this matters now: Students increasingly depend on digital academic resources, and a centralized platform can reduce the time spent searching for material while encouraging students to contribute resources to a shared academic community.

4 Proposed Solution . . . Software Proposition

4.1 Overview

Build a web-based **Collaborative Study Platform** with the following components:

- **Student authentication:** Users can create accounts and sign in to access platform features.
- **Notes repository:** Students can upload and access academic notes organized using information such as year, stream, and subject code.
- **Books repository:** Reference books and other academic resources can be uploaded and accessed through the platform.
- **PYQ repository:** Previous-year question papers can be organized and shared for examination preparation.
- **Class creation and participation:** Students can create or join classes/study groups to facilitate collaborative learning.
- **Resource categorization:** Academic resources are associated with relevant metadata such as academic year, stream, subject code, and resource type.
- **File storage:** Uploaded files can be stored using cloud object storage rather than relying entirely on local application storage.
- **Backend API:** A Go-based backend manages authentication, resource operations, database interactions, and file-upload workflows.
- **Database:** PostgreSQL stores structured application data including users and academic resource metadata.
- **Frontend interface:** A React-based frontend provides the user-facing interface for authentication, resource discovery, classes, and uploads.

4.2 Core Workflow

1. A student creates an account or signs into the platform.
2. The authenticated student accesses the study platform and selects the required resource category such as Notes, Books, or PYQs.
3. The student selects relevant academic information such as year, stream, and subject.
4. The system communicates with the backend API to retrieve matching resources from the database.
5. A student who wants to contribute material provides the required metadata and selects the file for upload.
6. The backend validates the request and stores the resource metadata in PostgreSQL while the file can be stored using cloud object storage.
7. The uploaded resource can pass through the application's resource-status workflow before becoming available for general access.
8. Other students can subsequently discover and access the resource through the appropriate section of the platform.
9. Students can additionally create or join classes to support collaborative study activities.

4.3 Operational Constraints for an Educational Setting

- Keep the platform usable on low-to-mid-range devices through responsive web design.
- Avoid unnecessary dependence on advanced research algorithms in the initial version; focus on workflow correctness, usability, security, and reliable resource management.
- Use mature technologies such as React, Go, PostgreSQL, and cloud object storage rather than developing custom infrastructure.
- Keep academic resource metadata structured so that searching and filtering can be improved as the platform grows.
- Ensure that authentication is required for protected operations.
- Store large files externally through object storage where appropriate rather than making the application server responsible for permanently storing all uploaded content.

5 Solution Approach . . . Engineering Focus

This is an engineering course project, so the primary focus is on building a reliable collaborative academic-resource platform rather than proposing novel research algorithms.

The system combines a responsive frontend, REST-style backend services, relational database storage, authentication, and cloud-based file storage.

5.1 Resource Organization ... Structured Academic Access

Academic resources will be organized using metadata such as:

- Academic year
- Stream
- Subject code
- Resource category
- File name
- Upload information
- Resource status

This organization allows the platform to retrieve relevant resources based on the student's academic requirements.

Example Parameter Mapping:

- **Year:** 3
- **Stream:** Computer Science
- **Subject Code:** UML501
- **Resource Type:** Notes

The system can then retrieve resources corresponding to these parameters rather than requiring students to manually search through unrelated files.

5.2 Web Architecture ... Web-based Solution

A typical 3-tier web architecture will be used:

- **Frontend:** React-based responsive user interface for authentication, resource browsing, resource uploads, class participation, and other student interactions.
- **Backend API:** Go-based backend responsible for authentication, authorization, resource operations, database communication, and file-upload workflows.
- **Database:** PostgreSQL stores users, resource metadata, class-related information, and other structured application data.
- **Object Storage:** Cloud object storage such as AWS S3 is used for uploaded academic files where required.

The architecture separates frontend, backend, database, and file-storage responsibilities, allowing individual components to be developed and scaled independently.

5.3 Fast Delivery and Deployment Enabler

The project will use incremental development to reduce release friction.

Development will prioritize:

- Running and validating the frontend independently.
- Running and validating the backend independently.
- Testing API communication between frontend and backend.
- Testing database operations.
- Testing file-upload functionality.
- Verifying authentication and protected routes.
- Maintaining a Git-based version history.

A stable version of the project will be maintained before major feature modifications so that experimental changes can be isolated from the working version.

6 Evaluation Criterion ... Measurable and Attributable

The success of the platform will be evaluated using measurable criteria connected to the core student workflow.

6.1 Primary Evaluation Metric

Time-to-resource-access (TRA): The time required for a student to locate and access a required academic resource after opening the platform.

- **Target:** The platform should allow a student to reach a relevant resource through structured navigation within a short interaction sequence.
- **Attribution:** The process can be evaluated through controlled user tasks and application-level timestamps/logging where applicable.

6.2 Secondary Evaluation Metrics

- **Resource discovery:** Percentage of test users who successfully locate a requested note, book, or PYQ.
- **Upload success rate:** Percentage of valid resource uploads successfully stored and registered by the system.
- **API reliability:** Percentage of tested API requests completing successfully without server-side failure.
- **User clarity:** Student-reported ease of finding and accessing academic material using a simple 1–5 feedback scale.
- **Authentication reliability:** Successful login/signup rate for valid credentials.
- **Resource organization:** Percentage of uploaded resources containing the required academic metadata.
- **Platform availability:** Availability of the frontend and backend during the testing/pilot period.

6.3 Pilot Validation Plan ... High-level

1. Recruit approximately 10–20 student users for initial testing.
2. Provide users with representative tasks such as:
 - Finding a specific subject’s notes
 - Accessing a PYQ
 - Locating a book
 - Uploading a note
 - Joining/creating a class
3. Record task completion and failures.
4. Collect short user feedback regarding navigation, resource discovery, upload experience, and overall usability.
5. Compare the results across iterations and prioritize improvements based on observed problems.

7 Scalability ... With Foresight

The platform should scale both theoretically and practically as the number of students and academic resources increases.

7.1 Scalable Theoretically

The system can support increasing numbers of resources and users through:

- Decoupling the frontend and backend
- Using a relational database for structured data
- Indexing commonly queried fields
- Pagination for large resource lists
- Stateless API design where appropriate
- External object storage for large files

7.2 Operationally Deployable Practically

The platform can be deployed using common web-hosting and managed infrastructure:

- Managed PostgreSQL database
- Cloud object storage
- Hosted frontend
- Hosted backend/API
- Environment-based configuration
- Git-based version control
- CI/CD pipeline in later iterations

This allows the application to move beyond a local development environment without requiring a complete redesign.

8 Engine Availability Heuristic

A mature ecosystem of tools and services is available to implement the proposed platform:

- **Frontend framework:** React / TypeScript
- **Backend framework:** Go with a web/API framework
- **Relational database:** PostgreSQL
- **Authentication:** Token/session-based authentication mechanisms
- **Object storage:** AWS S3 or equivalent cloud storage
- **HTTP communication:** REST-style API communication between frontend and backend
- **Version control:** Git and GitHub
- **Testing:** Frontend and backend testing frameworks can be incorporated as the project matures
- **Deployment:** Common cloud hosting and CI/CD services can be used for staging and production

We will use these mature components to focus on collaborative learning workflows, resource organization, correctness, usability, and scalability rather than developing new infrastructure.

9 Project Scope and Deliverables ... Iteration-friendly

9.1 Initial Deliverable ... First Iteration

The initial working version includes:

- Student registration and login
- Authenticated student dashboard
- Notes section
- Books section
- PYQ section
- Academic resource categorization
- Resource upload workflow
- Class creation and participation functionality
- Backend API for frontend communication
- PostgreSQL database for structured application data
- Cloud storage integration for uploaded files
- Basic authentication and authorization
- Basic resource status handling
- Working frontend-backend integration

9.2 Subsequent Deliverables

Potential subsequent iterations can include:

- Improved search and filtering
- Resource ratings and quality indicators
- Comments and discussions on resources
- Notifications for new resources and class activity
- Contributor profiles and contribution statistics
- Administrative moderation dashboard
- Improved resource recommendation
- Analytics for popular subjects and resources
- AI-assisted PDF summarization
- Question answering over uploaded academic material
- Automatic quiz generation from notes
- AI-based academic resource recommendations

These later features can be introduced incrementally without changing the fundamental architecture.

10 Risks and Mitigations

- **Data privacy:** Minimize collection of unnecessary personal information and ensure that authentication and access-controlled operations are properly implemented.
- **Unauthorized resources:** Introduce resource validation/moderation workflows to reduce the possibility of inappropriate or irrelevant material being published.
- **Storage dependency:** Use appropriate cloud-storage configuration and maintain clear separation between application metadata and uploaded files.
- **AWS configuration failures:** Keep cloud credentials and configuration outside source code and use appropriate environment-based configuration.
- **Large files:** Avoid committing uploaded academic files or dependency directories into the Git repository. Store application dependencies through package manifests and academic uploads through the intended storage system.
- **User adoption:** Keep resource discovery and upload workflows simple and minimize unnecessary steps.
- **Data quality:** Require essential metadata such as year, stream, subject code, and resource type before accepting a resource.
- **Scalability:** Use database indexing, pagination, separated frontend/backend components, and external object storage as resource volume increases.
- **Operational issues:** Maintain a stable Git version of the working project before introducing experimental modifications.

11 Summary

This project proposes a deployable web-based **Collaborative Study Platform** designed to reduce the time and effort students spend searching for and sharing academic resources.

The platform combines student authentication, structured academic-resource management, notes, books, previous-year questions, class participation, file uploads, a React frontend, a Go backend, PostgreSQL database storage, and cloud-based file storage.

It meets the course-oriented engineering objective by emphasizing practical workflow design, incremental development, maintainability, measurable evaluation, and scalability readiness rather than research-driven novelty.

The project can initially provide a reliable centralized platform for academic resource sharing and can subsequently evolve toward advanced capabilities such as recommendation systems, analytics, notifications, and AI-assisted learning.

The primary value of the system is therefore not simply providing another LMS, but creating a **student-driven collaborative academic ecosystem** in which students can both consume and contribute study resources.